CM3035 Final Report: Arete, an eLearning Application

1. Introduction, and the Shape of the Application

Arete, an eLeanring application, is designed to allow for a series of courses to be broken down into per term offerings. Then a teacher is attached to each offering. The teahcer of the offering is able to publish course and teaching mateiral. Each offering also has a cohert of students enrolled. Students can enroll in any offering via the cataloge page. The students can give feedback on the offering. The teacher of the offering is able to remove any student in there course for temporary or perminant status. Both students and teachers are able to join a chat app, with rooms per offering. Finally, their is a status blog post, the users can use and other users can see when they visit other users pages.

The basic design of the application is two parts. The main eLearning app is `arete`. Which is the main models, templates, and permission authorisation logic for users. While `chat` is the asynchronous live chat feature. The chat lives on the detail pages per offering. This page is gated to only the enrolled students and the teacher teaching the course. Thus only the users associated to the offering can talk to one another.

2. The Data Model

To meet the requirements I designed ten models for the application.

First is `AppUser`, it expands the default django `User` with ability to attach a profile photo to each user. This photo is stored in the `media/` directory, not with the static files that ship with the application. The model stores the path, and the upload is sanitised for us: Django cleans the filename so it cannot escape that directory, and because the field is an `ImageField` behind a `ModelForm`, Pillow opens the file and rejects anything that is not a genuine image. Next, `Course`, is the parent to the heart of the application `Offering`. I split the two because I felt it best served the eLearning environment. Students and teachers are not linked to a course, instead they enrol and teach the offering. The offering is the instance of a course with information that changes every year. While `Course` is more a constant that would be administered by a department head. A unique constraint on `(course, season, year)` keeps that split honest: the database itself refuses two runnings of the same course in one term. `Enrolment` is the many-to-many link between `Students` and `Offerings`. Each offering has a cohort of students, and a student needs to be able to enrol in multiple courses to finish a degree. Those are the main models. Next are models that add information to the user or offering.

A teacher is an owner of an offering. There is no `Teacher` table: a teacher is a `User` in the Teachers group, and `Offering.teacher` is the foreign key that makes them the owner of that one offering. Only one teacher has this responsibility per `Offering`. As the owner, they have certain privileges. A teacher can add files to the offering as `Materials`, which like the profile photo are a stored path and a file in `media/`. They can also set deadlines as `Assignments`, which hold a title, a description and a due date rather than a file. In the current product, the model

allows a teacher to create an assignment for an offering. This is shown in the script to populate the data. However, I ran out of time to create a view that allows this feature. Thus assignments and their deadlines are only created via the admin or the seed data. When the teacher uploads material, every actively enrolled student is notified of the event via `Notification`. The teacher can remove or block students from their offering, in which case they are skipped. This is a main feature for `Enrolment`. A notification stores its recipient, the offering, a link, a read flag and the message itself. Notifications appear on the user's homepage, and one button marks them all read, after which they are no longer included in the context. Every actively enrolled student can leave feedback on their offering from the offering detail page, a written comment with an optional 1 to 5 rating, once per offering. Any logged in user, on their home page, has a status update blog, where they can comment on an offering they are a part of, or a general posting.

Finally every offering has its own chat room. The page is available from the offering page, so a teacher and the students of that course have somewhere to talk in a live asynchronous conversation. The room is a name in the URL rather than a table, `room3` for offering 3, and the consumer requires the visitor to be logged in. It does not yet check that they belong to the offering. This means only the links to the chat rooms are authenticated not rooms themselves.

2.1 `Course` and `Offering`

One of the earliest design decisions I made was to split a course into two. This is because the cataloging of classes and the running of those classes are different things. `Course` is MATH101 "Discrete Mathematics". This will always stay the same. `Offering` is MATH101 in Winter 2024, taught by Mr Smith. An offering will change for the three different terms per year, every year.

```
class Offering(models.Model):
    class Season(models.TextChoices):
        WINTER = "WINTER", "Winter"
        SPRING = "SPRING", "Spring"
        SUMMER = "SUMMER", "Summer"
    season = models.CharField(max_length=16, choices=Season.choices, ...)
    year = models.IntegerField(null=False, blank=False)
    course = models.ForeignKey(Course, on_delete=models.CASCADE,
related_name='offerings')
    teacher = models.ForeignKey(User, on_delete=models.CASCADE,
related_name='offerings_taught')
    students = models.ManyToManyField(User, through='Enrolment',
related_name='offerings_taken')
```

If we had left Course as "MATH101 in Winter 2024, taught by Mr Smith with this cohort of students", every new term, year or change of teacher would result in a new MATH101. Also, it would be impossible to distinguish between the different MATH101 classes. Or if we had just the course and then updated it per term or teacher change, we would lose the history of the course.

Next was deciding how to link the students to the course. The many to many link table `Enrolment` says "studentA is enrolled in MATH101 and is in active status". The status allows us to remove and ban a student from the offering.

2.2 Normalisation

First normal form

Every column holds one atomic value. The easy violation here would be a list of students in an offering. We prevented this with the enrolment table. An intial draft, of the models, did violate 1nf by storing the term as a composite of season (winter, summer, spring) and year.

Second normal form

A table is in 2NF when it is in 1NF and every non-key column depends on the whole of its key, not just part of it, so this rule can only be violated with models that have a compound primary key.

Enrolment
offering (pk)
student (pk)
status
enrolled_at

Both `status` and `enrolled_at` relie on know what student in what class. With out both keys the other columns have no meaning. Thus they relie on both keys.

Feedback
offering (pk)
student (pk)
rating
comment

`Feedback` Is the exact same as `Enrolment`. An offering can't reveiw itself and a student needs something to be reviewed. Therefore it too is 2nf.

Finally is the most complex model `Offering`

Offering
course (pk)
season (pk)
year (pk)
teacher
students

The earlier decicion to split offering form course, as gut move, acutaly put the model in 2nf. Without that split, `title` and `description` would depend on `code` alone. Now assuming a

diverse bench of teachers that get rotated out teacher relies on knowing all three pks. However, I just relised, that is an assumption for an in person course, not an elearning environment. Like this course which is recored in 2020 and then every term has the same teacher. However, it does have differnt tutors.

Third normal form

Every non-key column depends entirly on the primary keys, with no transitive dependies. The notification violates this becasue it stores the name of the sender and recipient as strings ranther than linking the users. This means that if a user changes their name, the strings will point to usernames that dont exisit. While this might be an oversight, the goal was to show who was sending it at the moment, as notifications have short lives.

3. Accounts and Roles: R1a, R1b, R1c

3.1 Creating an account and logging in (R1a, R1b)

Django has a defualt user class, `django.contrib.auth`. It provides basic functionality for users, groups, permissions, password hashing, and the functions to log in and log out. I extened the basic user with `AppUser` which allows the user to upload a profile picture.

```
class AppUser(models.Model):
    # Base django user model
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    # profile photo
    photo = models.ImageField(null=True, blank=True)
```

The `User` is just a model. So my initial design of two seperat users `Student` and `Teacher` tables. Resulted in the need for seperate passwords and logins. And other functionality like `request.user` no longer responed like in the lectures. Thus I moved the type of user to the registration forms.

Registration saves two forms at once. `UserForm` creates the `User` and sets the password. While `UserProfileForm` creates the matching `AppUser` and asks "I am a" student or teacher. Then I pre made two groups with unique permission expressing laid out, and the view then adds the new user to that group.

apps.py

```
# Group permissions
TEACHER_PERMISSIONS = [
    'add_course', 'change_course',
    'add_offering', 'change_offering',
    'add_material',
    'add_assignment',
    'change_enrolment',        # R1h: remove or block a student
    'add_statusupdate',        # R1i says users, not only students
```

```
    'view_appuser',            # R1c: search for students and teachers
]

STUDENT_PERMISSIONS = [
    'add_enrolment',           # R1e: enrol on an offering
    'add_feedback',            # R1f
    'add_statusupdate',        # R1i
]
```

arete/veiw.py

```
def register(request):
    """
    Register view for handling user registration.
    - Successful registration redirects to the login page.
    - Invalid form data returns error messages.
    """

    registered = False

    # User has sent us data (filled in registation forum)
    if request.method == 'POST':
        user_form = UserForm(data=request.POST)
        profile_form = UserProfileForm(data=request.POST,
files=request.FILES)

        # The users forms are correct
        if user_form.is_valid() and profile_form.is_valid():

            # create the django user
            user = user_form.save(commit=False)
            user.set_password(user.password)
            user.save()

            # create the profile object
            profile = profile_form.save(commit=False)
            profile.user = user
            profile.save()

            # We dont sanitize the photo path here for AppUser.photo because
the models data type ImagField handles it.

            # apply the group for the profile permissions
            group = Group.objects.get(
                name=profile_form.cleaned_data['group'])
            user.groups.add(group)

            # Flip the proflie flag
            registered = True
```

3.2 Students and teachers

As seen above, Teachers and Students have different permissions. However, a permission only answers the question "what are you allowed to do?". It does not answer "who are you?". So a

protected page in the application asks up to three questions, in order:

1. **Are you logged in?** If not, you are redirected to the login page.
2. **Are you allowed to do this?** This is the permission. A student who tries to add material does not hold `add_material`, so they get a 403.
3. **Is it yours?** Only the teacher of that offering may add material to it, so a different teacher, who does hold `add_material`, is turned away too.

The task of creating a `Material` for an offering is an example that ask asks all three of these questions. Django answers the first two for me, and the third I had to write myself.

For the question permissions cannot answer, "who are you?", I added two helpers in `arete/permissions.py` that check group membership instead:

```
def is_teacher(user):
    return user.groups.filter(name=TEACHERS).exists()
```

So I ended up using both, for different jobs:

- **Permissions decide who can reach a page or take an action.**
  - `CourseCreate` needs `add_course`
  - `FeedbackCreate` needs `add_feedback`
- **Group checks decide what someone is.** `is_teacher()` and `is_student()`. These functions were neccasary for the home template page. Where the user sees different things depending on who they are. Students need to see the upcoming deadlines, while teachers create assingments and search for students.

That split is not applied consistently yet, and there are parts of it I do not like. The teacher dashboard and the Teaching link in the navigation are gated on `add_offering`, which is a permission standing in for "is a teacher".

3.3 Only your own offering

A permission applies to the whole group, not to one user. The Teachers group grants every one of its users the permission `change_enrolment`, so without another check, any teacher could change the enrolment status of any student on any offering. I wanted only the teacher teaching an offering to be able to change the enrolment status of their own students. Therefore I needed another check, a combination check of the user and resourse.

This is question 3 from section 3.2, "is it yours?". the `offering_detail` page uses `has_permission()`. It first calls `super().has_permission()`, which checks the users group permission. Once checked, and filtered out students, it then compares the logged-in user with the offering's teacher:

```
def has_permission(self):
    """
    Only the Teacher of the offering or superuser has access
    """
    if not super().has_permission():
        return False
    user = self.request.user
```

```
    return self.get_object().offering.teacher == user or user.is_superuser
```

The same pattern guards three views:

- `MaterialCreate` needs `add_material`, and the offering must be the teacher's own.
- `FeedbackCreate` needs `add_feedback`, and the student must be actively enrolled on that offering.
- `EnrolmentStatusUpdate` needs `change_enrolment`, and the enrolment must be on the teacher's own offering, or the user must be the administrator.

3.4 Searching for users (R1c)

Only teachers can search for other users. It is under a `is_teacher()` check. It renders on their home page. The search works by matching usernames, first and last names.

```
return User.objects.filter(
    Q(is_active=True) & (
        Q(username__icontains=cleaned_query) |
        Q(first_name__icontains=cleaned_query) |
        Q(last_name__icontains=cleaned_query)
    )
).distinct()
```

The return is links as unorder lists, linking the the users home page.

4. Critical Evaluation

4.1 Future development

- **A course does not have to become an offering.** Creating a course redirects straight to the offering form; better to ask, and prefill the course if the answer is yes.
- **Students cannot drop an offering.** A POST setting their own `ACTIVE` enrolment to `REMOVED` would fix it without letting a blocked student clear a block.
- **The catalogue does not sort or filter.** Within a year the seasons sort alphabetically rather than by calendar, past terms are listed, and `Enrol` does not check the year, so a student can enrol on an offering that has already run.
- **Full CRUD through the API.** Four endpoints are GET-only. Lectures 4.2.4 to 4.2.7 and the midterm's `IncidentDetail` show the four mixins that would add POST, PUT and DELETE. Not built because each write endpoint would also need the access rules the HTML views enforce, repeated as DRF permission classes with tests for the refusals, and untested endpoints are worse than absent ones.
- **Assignments have no creation page**, so deadlines come only from the admin and the seed data, and the `add_assignment` permission is never checked. R1 does not ask for assignments at all.
- **The offering page does not show its own assignments**, so a teacher never sees an offering's deadlines in the application. Prototyped in a scratch copy on 12 September: the existing tests stayed green, and a new test failed correctly when the lists were not filtered by offering.

5. Running the Application

5.1 Install and run

```
unzip elearning.zip && cd elearning
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

sudo service postgresql start && sudo service redis-server start
createdb elearning_db

python manage.py migrate
python manage.py createsuperuser
python scripts/populate_elearning.py
python manage.py runserver
```

Then open `http://127.0.0.1:8000/`

9.6 Login credentials

Role	**Username**	**Password**
Administrator	`admin`	`GpuDnjNPEP1QaY` (deployed instance; locally, use `createsuperuser`)
Teachers	`jsmith`, `lcarter`, `rhoover`	`password`
Students	`student01` to `student12`	`password`

9.7 Deployment (D4)

http://ec2-16-58-223-96.us-east-2.compute.amazonaws.com/